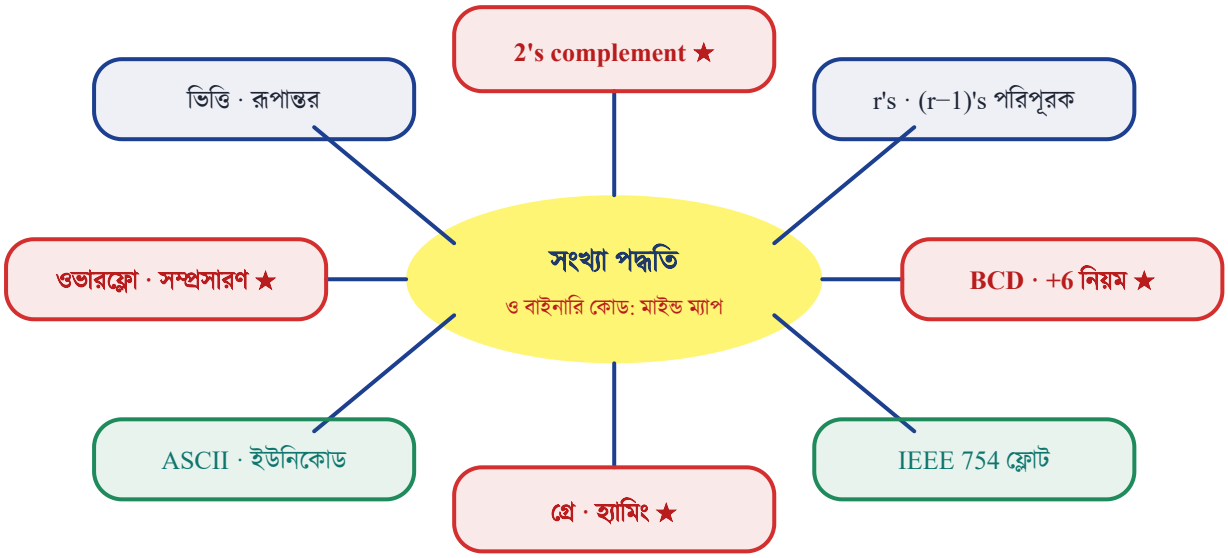
সংখ্যা পদ্ধতি ও কোড

পরিপূরক · ওভারফ্লো · BCD · গ্রে · হ্যামিং · IEEE 754: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = প্রয়োগ / ডেটা ফরম্যাট

সিলেবাসের যে অংশ

"Number system: **binary, octal, hexadecimal**, binary arithmetic" (কম্পিউটার ফান্ডামেন্টালস) এবং "Digital logic: … **Arithmetic and data handling logic circuits**" (DLD)। ভিত্তি সংক্ষেপে; গভীরতা **চিহ্নযুক্ত সংখ্যা, পরিপূরক, ওভারফ্লো**, বাইনারি কোড, ত্রুটি সংশোধন কোড ও ফ্লোটিং পয়েন্টে। পূর্ণ গাণিতিক অংশ কম্পিউটার ফান্ডামেন্টালসের "সংখ্যা পদ্ধতি ও বাইনারি গণিত" নোটে।

হাই-ইল্ড তালিকা

① 2's complement-এর পরিসর
② −N বের করার শর্টকাট
③ ওভারফ্লো নিয়ম (V ফ্ল্যাগ)
④ BCD: > 9 হলে 6 যোগ
⑤ বাইনারি ↔ গ্রে
⑥ হ্যামিং প্যারিটি বিট
⑦ IEEE 754 সিঙ্গেল

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ ভিত্তি ও রূপান্তর ..... পৃ. ২
✓ চিহ্নযুক্ত সংখ্যার উপস্থাপন ..... পৃ. ৩
✓ 2's complement, ওভারফ্লো ..... পৃ. ৪
✓ r's / (r−1)'s পরিপূরক ..... পৃ. ৫
✓ দশমিক কোড ও গ্রে কোড ..... পৃ. ৬
✓ BCD যোগ, ASCII, ইউনিকোড … পৃ. ৭
✓ প্যারিটি ও হ্যামিং কোড ..... পৃ. ৮
✓ IEEE 754 ফ্লোটিং পয়েন্ট ..... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ..... পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

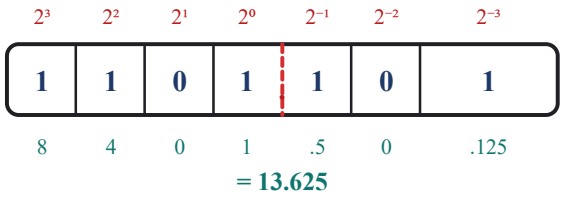
১ স্থানিক সংখ্যা পদ্ধতি

সংজ্ঞা: ভিত্তি (radix) **r**-এর **স্থানিক (positional, weighted)** পদ্ধতিতে অঙ্ক 0 … r − 1; i-তম স্থানের মান r^i: $\mathbf{N = \Sigma\, d_i \cdot r^i}$ (ভগ্নাংশে $i < 0$)। বাঁয়ের বিট = **MSB**, ডানের বিট = **LSB**।

পদ্ধতি	ভিত্তি	অঙ্ক	উদাহরণ	কোথায় দেখি
বাইনারি	2	0, 1	$(1011)_2 = 11$	সব হার্ডওয়্যার, বিট মাস্ক
অক্টাল	8	0 – 7	$(157)_8 = 111$	ইউনিক্স পারমিশন: chmod 755
ডেসিমাল	10	0 – 9	$(111)_{10}$	মানুষের ইনপুট/আউটপুট
হেক্সাডেসিমাল	16	0 – 9, A – F	$(6F)_{16} = 111$	অ্যাড্রেস, MAC, রং #FF8800

১ নিবল = 4 বিট · ১ বাইট = 8 বিট · n বিটে **2^n টি বিন্যাস**; N টি মানের জন্য $\lceil \log_2 N \rceil$ বিট লাগে (যেমন ১০০০টি মান → ১০ বিট)।

প্রচলিত ভুল ধারণা: "সব সংখ্যা পদ্ধতি স্থানিক": **ভুল**। রোমান সংখ্যা (XIV) অস্থানিক। কম্পিউটারের চারটি পদ্ধতি স্থানিক।



$(1101.101)_2$-এর স্থানীয় মান

২ এক পাতায় সব রূপান্তর

ভিত্তি r → ডেসিমাল

প্রতিটি অঙ্ককে তার স্থানীয় মান দিয়ে গুণ করে যোগ করো।

$(3A.C)_{16} = 3\times16 + 10 + 12/16 = \mathbf{58.75}$

$(101.11)_2 = 4 + 1 + 0.5 + 0.25 = \mathbf{5.75}$

ডেসিমাল → ভিত্তি r

পূর্ণ অংশ: r দিয়ে বারবার ভাগ, ভাগশেষ **নিচ থেকে ওপরে** পড়ো।

ভগ্নাংশ: r দিয়ে বারবার গুণ, পূর্ণ অংশ **ওপর থেকে নিচে** পড়ো।

সমাধানকৃত উদাহরণ ✍️: $(235.625)_{10}$ → বাইনারি

÷ 2	235	117	58	29	14	7	3	1	× 2	.625	.25	.5
ভাগশেষ	1	1	0	1	0	1	1	1	পূর্ণ	1	0	1

ভাগশেষ ডান → বাঁ (শেষেরটি আগে): 11101011; ভগ্নাংশ বাঁ → ডান: .101 → $\mathbf{(11101011.101)_2} = (353.5)_8 = (EB.A)_{16}$

বাইনারি ↔ অক্টাল ↔ হেক্স: বিন্দু থেকে বাইরের দিকে গ্রুপ, ফাঁকা হলে 0

$(157)_8$ → 001 101 111 → 0110 1111 → $\mathbf{(6F)_{16}}$ · অক্টাল ↔ হেক্স যায় **বাইনারি হয়ে**

মনে রাখার কৌশল: "**অক্টাল ৩, হেক্স ৪**": $8 = 2^3$, $16 = 2^4$ বিট প্রতি অঙ্কে। $2^{10} = 1024$, $2^{16} = 65\,536$ মুখস্থ রাখো।

পরীক্ষার ফাঁদ: $(0.1)_{10} = 0.000110011\ldots_2$ **কখনো শেষ হয় না**; শুধু $k/2^m$ আকারের ভগ্নাংশ বাইনারিতে সসীম।

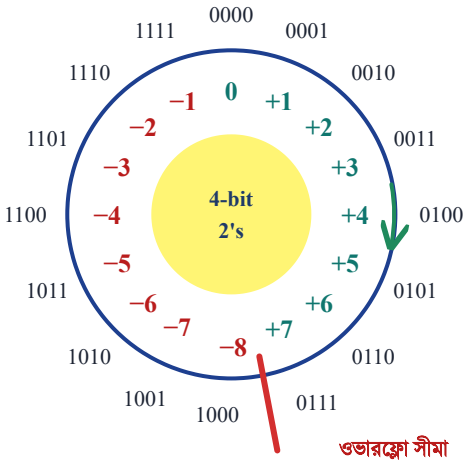
৩ চিহ্নযুক্ত সংখ্যার উপস্থাপন

সংজ্ঞা: হার্ডওয়্যারে "−" চিহ্ন নেই, তাই **MSB-কে চিহ্ন বিট (sign bit)** ধরা হয় (0 = +, 1 = −)। ধনাত্মক সংখ্যা তিন পদ্ধতিতেই একই; পার্থক্য শুধু −N লেখায়:

- **চিহ্ন-মান (Sign-magnitude, SM):** চিহ্ন বিট 1, মান অপরিবর্তিত
- **১-এর পরিপূরক (1's complement):** +N-এর প্রতিটি বিট উল্টাও (= $(2^n - 1) - N$)
- **২-এর পরিপূরক (2's complement):** বিট উল্টে 1 যোগ (= $\mathbf{2^n - N}$)

সমাধানকৃত উদাহরণ ✍️: 8 বিটে −45 (+45 = 0010 1101): SM = **1010 1101** · 1's = **1101 0010** · 2's = 1101 0010 + 1 = **1101 0011**

বিট	চিহ্নহীন	SM	1's	2's
0000	0	+0	+0	0
0001	1	+1	+1	+1
0011	3	+3	+3	+3
0111	7	+7	+7	**+7 (সর্বোচ্চ)**
1000	8	**−0**	−7	**−8 (সর্বনিম্ন)**
1001	9	−1	−6	−7
1011	11	−3	−4	−5
1100	12	−4	−3	−4
1110	14	−6	−1	−2
1111	15	−7	**−0**	−1



2's complement চাকা: +1 = ঘড়ির কাঁটার দিকে এক ধাপ

পদ্ধতি (n বিট)	পরিসর	8-বিট	শূন্য
চিহ্নহীন (unsigned)	$0 \dots 2^n - 1$	0 … 255	একটি
চিহ্ন-মান	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**দুটি (±0)**
1's complement	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**দুটি (±0)**
2's complement	$\mathbf{-2^{n-1} \dots +(2^{n-1} - 1)}$	**−128 … +127**	একটি

2's complement 16-বিট: **−32 768 … +32 767** · 32-বিট (C *int*): $\mathbf{-2^{31} \dots 2^{31} - 1}$

পরীক্ষার ফাঁদ: $-2^{n-1} \dots 2^{n-1} - 1$ পরিসর **শুধু 2's complement-এর**; SM ও 1's complement-এ একটি বিন্যাস −0-তে যায়, তাই সেগুলো $-(2^{n-1} - 1)$-এ থামে।

মনে রাখার কৌশল: 2's complement-এ **একটি ঋণাত্মক সংখ্যা বেশি**: −128 = 1000 0000-এর কোনো +128 জোড়া নেই; একে ঋণাত্মক করলে আবার নিজেকেই পাওয়া যায়।

৪ সব CPU কেন 2's complement ব্যবহার করে

- **একটিই শূন্য** (±0 যাচাইয়ের ঝামেলা নেই)
- **A − B = A + B′ + 1**: একই অ্যাডার
- MSB-এর ক্যারি আউট শুধু ফেলে দাও
- চিহ্ন সম্প্রসারণ = MSB কপি

ভারযুক্ত দৃষ্টিভঙ্গি: MSB-এর মান -2^{n-1}:
1101 0011 = −128 + 64 + 16 + 2 + 1 = **−45**

মনে রাখার কৌশল: শর্টকাট: ডান দিক থেকে: **প্রথম 1 পর্যন্ত (সেটিসহ) বিট হুবহু লেখো, বাকিগুলো উল্টাও**: 0110 1000 (104) → 1001 1000 (−104) ✓

৫ চিহ্ন সম্প্রসারণ (Sign Extension)

2's complement সংখ্যাকে বেশি বিটে নিতে নতুন প্রতিটি উচ্চ বিটে **চিহ্ন বিট কপি** করো; মান বদলায় না।

−5: 1011 → **1111 1011** · +5: 0101 → 0000 0101

(x86: *MOVSX* চিহ্ন-সম্প্রসারণ, *MOVZX* শূন্য-সম্প্রসারণ)

পরীক্ষার ফাঁদ: ঋণাত্মক সংখ্যার সামনে 0 বসালে মান বদলে যায়: 1011 (−5) → 0000 1011 = **+11**। শূন্য-সম্প্রসারণ শুধু চিহ্নহীন সংখ্যার জন্য।

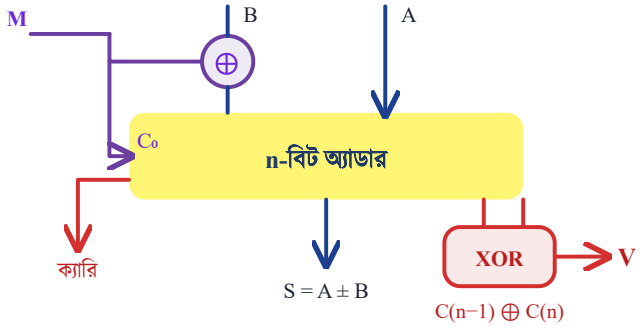
৬ ওভারফ্লো শনাক্তকরণ

নিয়ম: ওভারফ্লো: **একই চিহ্নের** দুটি সংখ্যা যোগে ফল **বিপরীত চিহ্নের** হলে (ভিন্ন চিহ্নে কখনো ওভারফ্লো হয় না)। হার্ডওয়্যারে: $V = C_{n-1} \oplus C_n$, অর্থাৎ MSB-**তে আসা** ক্যারি XOR MSB **থেকে বের হওয়া** ক্যারি।

8-বিট যোগ	বাইনারি	ফলের বিট	C_{in} C_{out}	V	অর্থ
70 + 80	0100 0110 + 0101 0000	1001 0110	1 0	**1**	−106 পড়ায়: ওভারফ্লো
−70 + (−80)	1011 1010 + 1011 0000	(1) 0110 1010	0 1	**1**	+106 পড়ায়: ওভারফ্লো
45 + (−20)	0010 1101 + 1110 1100	(1) 0001 1001	1 1	0	+25 ✓ ক্যারি বাদ
100 + 27	0110 0100 + 0001 1011	0111 1111	0 0	0	+127 ✓ ঠিক আঁটে

পরীক্ষার ফাঁদ: **ক্যারি ≠ ওভারফ্লো**। ক্যারি আউট (C ফ্ল্যাগ) চিহ্নহীন ওভারফ্লো বোঝায়; V ফ্ল্যাগ চিহ্নযুক্ত ওভারফ্লো। ৩য় সারিতে ক্যারি আছে, ওভারফ্লো নেই।

ADD-এর পর CPU ফ্ল্যাগ: **C** ক্যারি, **V/O** ওভারফ্লো, **N/S** চিহ্ন = MSB, **Z** শূন্য।



অ্যাডার-সাবট্রাক্টর: M = 0 যোগ, M = 1 বিয়োগ

৭ r's ও (r − 1)'s পরিপূরক

সংজ্ঞা: r ভিত্তির n-অঙ্কের N: **r's (radix)** = $r^n - N$ · **(r − 1)'s (diminished radix)** = $(r^n - 1) - N$ · r's = (r − 1)'s + 1

ভিত্তি	(r − 1)'s	r's	(r − 1)'s পাওয়ার উপায়	উদাহরণ
10	9's	10's	প্রতিটি অঙ্ক 9 থেকে বিয়োগ	546700 → 453299 / **453300**
2	1's	2's	প্রতিটি বিট উল্টাও	1101100 → 0010011 / **0010100**
8	7's	8's	প্রতিটি অঙ্ক 7 থেকে বিয়োগ	$(325)_8$ → 452 / **453**
16	15's	16's	প্রতিটি অঙ্ক F থেকে বিয়োগ	$(3A)_{16}$ → C5 / **C6**

মনে রাখার কৌশল: এক টানে 10's complement: ডানের শূন্যগুলো রেখে দাও, প্রথম অশূন্য অঙ্ক **10** থেকে, তার বাঁয়ের সব অঙ্ক **9** থেকে বিয়োগ: 546<u>00</u> → 453<u>00</u>।

৮ পরিপূরকের সাহায্যে বিয়োগ: M − N

r's পরিপূরক দিয়ে:

১. N-কে M-এর সমান অঙ্কে আনো
২. M + N-এর r's পরিপূরক
৩. **শেষ ক্যারি** → বাদ দাও (+)
৪. **ক্যারি নেই** → −(যোগফলের r's)

(r − 1)'s পরিপূরক দিয়ে:

১. M + N-এর (r − 1)'s পরিপূরক
২. **শেষ ক্যারি** → LSB-তে 1 যোগ (**এন্ড-অ্যারাউন্ড ক্যারি**)
৩. **ক্যারি নেই** → −(যোগফলের (r − 1)'s)

সমাধানকৃত উদাহরণ ১: ডেসিমাল ✍️: 72532 − 3250: 03250-এর 10's complement = 96750; 72532 + 96750 = **1**69282 → ক্যারি বাদ → **69282**।
3250 − 72532: 03250 + 27468 = 30718, **ক্যারি নেই** → −(30718-এর 10's complement) = **−69282**।

সমাধানকৃত উদাহরণ ২: বাইনারি ✍️: X = 1010100 (84), Y = 1000011 (67)

কাজ	হিসাব	ফল
X − Y, 2's	1010100 + 0111101 = **1** 0010001 → ক্যারি বাদ	**0010001 = 17**
X − Y, 1's	1010100 + 0111100 = **1** 0010000 → + 1 (এন্ড-অ্যারাউন্ড)	**0010001 = 17**
Y − X, 2's	1000011 + 0101100 = 1101111, ক্যারি নেই	**−0010001 = −17**
Y − X, 1's	1000011 + 0101011 = 1101110, ক্যারি নেই	**−0010001 = −17**

পরীক্ষার ফাঁদ: আগে N-এর সামনে **0 বসাও** (3250 → 03250)। 1's/9's-এ এন্ড-অ্যারাউন্ড ক্যারি ভুলে গেলে উত্তর **১ কম** হয়।

আইটি সংযোগ: ALU বিয়োগ করে B উল্টে ও carry-in = 1 দিয়ে: SUB আসলে 2's complement দিয়ে ADD (অ্যাডার-সাবট্রাক্টর, পৃ. ৪)।

৯ দশমিক অঙ্কের কোড (BCD পরিবার)

BCD (8421): প্রতিটি দশমিক অঙ্ক **আলাদাভাবে** 4 বিটে, ভার 8-4-2-1। ১৬টি বিন্যাসের মাত্র ১০টি ব্যবহৃত: **1010 – 1111 অবৈধ** (৬টি অব্যবহৃত)।

দশমিক	8421 (BCD)	2421 (Aiken)	Excess-3	84-2-1	গ্রে
0	0000	0000	0011	0000	0000
1	0001	0001	0100	0111	0001
2	0010	0010	0101	0110	0011
3	0011	0011	0110	0101	0010
4	0100	0100	0111	0100	0110
5	0101	**1011**	1000	1011	0111
6	0110	1100	1001	1010	0101
7	0111	1101	1010	1001	0100
8	1000	1110	1011	1000	1100
9	1001	1111	1100	1111	1101

- **ভারযুক্ত (weighted):** 8421, 2421, 84-2-1 (ভার ঋণাত্মকও হতে পারে)
- **ভারহীন: এক্সেস-৩ (Excess-3)** (= BCD + 0011), গ্রে
- **স্ব-পরিপূরক (self-complementing)** (বিট উল্টালেই 9's complement): **2421, Excess-3, 84-2-1**; 8421 **নয়**
- **একক-দূরত্ব / চক্রীয়:** গ্রে

উদাহরণ ✍️: 59 → বাইনারি 111011 (৬ বিট) · BCD **0101 1001** · XS-3 1000 1100

পরীক্ষার ফাঁদ: BCD ≠ বাইনারি: 59-এর BCD 0101 1001, 0011 1011 নয়। BCD-তে বিট বেশি লাগে, কিন্তু অঙ্কে রূপান্তর সহজ (ক্যালকুলেটর, মিটার, RTC চিপ)।

১০ গ্রে কোড (Reflected Binary)

পরপর দুটি মানে **ঠিক একটি বিট** বদলায় (শেষ → প্রথমেও)। ভারহীন: **গাণিতিক কাজে অচল**।

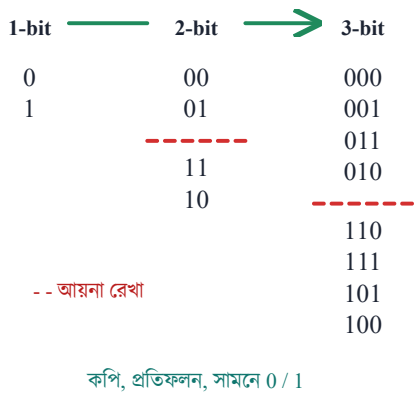
বাইনারি → গ্রে: $g_{MSB} = b_{MSB}$; $g_i = b_{i+1} \oplus b_i$ $(G = B \oplus (B >> 1))$

1011 → 1, 1⊕0, 0⊕1, 1⊕1 = **1110**

গ্রে → বাইনারি: $b_{MSB} = g_{MSB}$; $b_i = b_{i+1} \oplus g_i$ (সদ্য পাওয়া **বাইনারি** বিটের সাথে XOR)

1101 → 1, 1⊕1 = 0, 0⊕0 = 0, 0⊕1 = 1 → **1001**

ব্যবহার: K-ম্যাপের ক্রম, রোটারি/শ্যাফট এনকোডার, async FIFO পয়েন্টার।



মনে রাখার কৌশল: B → G: "প্রথমটি কপি, তারপর **পাশাপাশি** দুটির XOR"। G → B: "প্রথমটি কপি, তারপর **নতুন বাইনারি বিট** XOR পরের গ্রে বিট" (কোনাকুনি)।

১১ BCD যোগ: +6 সংশোধন

নিয়ম: BCD অঙ্ক দুটি সাধারণ বাইনারির মতো যোগ করো। 4-বিট যোগফল **> 9 (1010–1111) অথবা** ক্যারি বের হলে **0110 (6) যোগ করো** এবং ক্যারি পরের অঙ্কে পাঠাও। কেন 6? 16 − 10 = 6, অর্থাৎ ৬টি অবৈধ কোড ডিঙানো।

যোগ	বাইনারি যোগফল	বৈধ?	সংশোধন	BCD ফল
4 + 5	0100 + 0101 = 1001	হ্যাঁ (9)	লাগবে না	0000 1001 = 9
8 + 7	1000 + 0111 = 1111	> 9	+ 0110 → 1 0101	**0001 0101 = 15**
9 + 9	1001 + 1001 = 1 0010	ক্যারি	+ 0110 → 1 1000	**0001 1000 = 18**

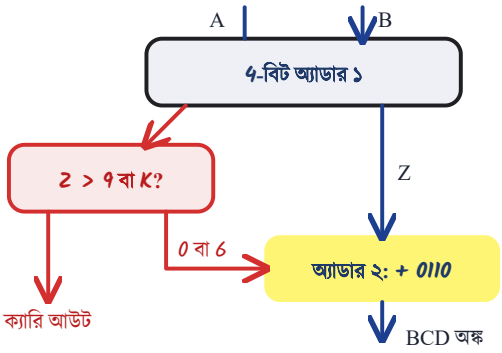
সমাধানকৃত উদাহরণ ✍️: BCD-তে 59 + 38

১. একক: 1001 + 1000 = 1 0001 → ক্যারি, তাই + 0110 = 0111, ক্যারি 1

২. দশক: 0101 + 0011 + 1 = 1001 (বৈধ)

৩. উত্তর **1001 0111** = 97 ✓

সংশোধন সংকেত: **C = K + Z_8Z_4 + Z_8Z_2** (K = ক্যারি আউট, Z = যোগফলের বিট)



একটি BCD অ্যাডার ধাপ (কম্বিনেশনাল সার্কিট)

পরীক্ষার ফাঁদ: 9 + 9 = 1 0010 **দেখতে** বৈধ (0010), কিন্তু ক্যারি আউটের কারণে + 6 লাগবেই। দুটি শর্তই দেখো। এক্সেস-৩ যোগে নিয়ম ভিন্ন: ক্যারি হলে +0011, না হলে −0011।

১২ আলফানিউমেরিক কোড

কোড	বিট	প্রতীক	মনে রাখো
ASCII	7	128	0–31 ও 127 কন্ট্রোল · 32 স্পেস · 95 মুদ্রণযোগ্য
Extended ASCII	8	256	যেমন ISO 8859-1 (Latin-1)
EBCDIC	8	256	IBM মেইনফ্রেম · 'A' = C1h · বর্ণ ধারাবাহিক নয়
ইউনিকোড	≤ 21	1 114 112	U+0000 … U+10FFFF · বাংলা **U+0980 – U+09FF**

ASCII: 'A' = **65** = 41h · 'a' = **97** = 61h · '0' = **48** = 30h · CR = 13, LF = 10, DEL = 127।

ইউনিকোডের এনকোডিং: **UTF-8** ১–৪ বাইট, ASCII-সামঞ্জস্যপূর্ণ (ওয়েবের ডিফল্ট) · UTF-16 ২ বা ৪ বাইট · UTF-

মনে রাখার কৌশল: **"A-65, a-97, 0-48"**: বড় ↔ ছোট হাতের পার্থক্য 32 = বিট 5 (0x20)। C-তে: *c ^ 0x20* কেস বদলায়, *'7' − '0'* = 7।

32 সবসময় ৪। 'অ' = U+0985 = UTF-8-এ **E0 A6 85** (৩ বাইট)।

পরীক্ষার ফাঁদ: মূল ASCII **7-বিট**, 8 নয়। ইউনিকোড হলো অক্ষর-সেট; UTF-8 তার একটি **এনকোডিং**।

১৩ প্যারিটি: ত্রুটি শনাক্তকরণ

সংজ্ঞা: প্যারিটি বিট মোট 1-এর সংখ্যাকে **জোড়** (even parity) বা **বিজোড়** (odd parity) করে। জেনারেটর/চেকার = **সব বিটের XOR** (XOR ট্রি, লজিক গেট নোট দেখো)।

ডেটা 1011001-এ চারটি 1 → জোড় প্যারিটি বিট **0**, বিজোড় প্যারিটি বিট **1**। জোড় প্যারিটিতে পাঠানো হয়: 10110010। **বিজোড় সংখ্যক** বিট উল্টালে ধরা পড়ে; ২, ৪ … টি ত্রুটি ধরা পড়ে না; অবস্থান নির্ণয় বা সংশোধন অসম্ভব।

পরীক্ষার ফাঁদ: একই বাইটে দুটি বিট উল্টালে প্যারিটি একই থাকে: **সাধারণ প্যারিটি তা ধরতে পারে না**। দ্বিমাত্রিক (সারি + কলাম) প্যারিটি একটি ত্রুটি সংশোধন করতে পারে।

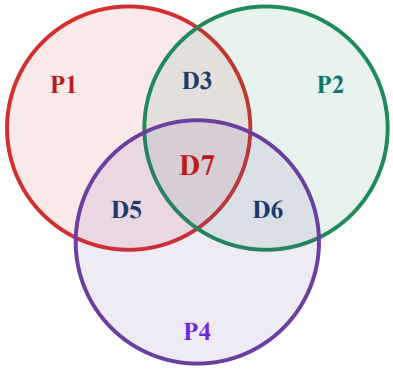
হ্যামিং দূরত্ব d: = দুটি কোড-শব্দ যত অবস্থানে ভিন্ন। ন্যূনতম দূরত্ব d_{min} হলে কোডটি সর্বোচ্চ $\mathbf{d_{min} - 1}$টি ত্রুটি **শনাক্ত** এবং $\mathbf{\lfloor (d_{min} - 1)/2 \rfloor}$টি **সংশোধন** করতে পারে। প্যারিটি: d = 2 (১টি শনাক্ত) · হ্যামিং(7,4): d = 3 (১টি সংশোধন) · SEC-DED: d = 4।

১৪ হ্যামিং কোড: একক ত্রুটি সংশোধন

প্যারিটি বিট সংখ্যা: ক্ষুদ্রতম p যাতে $\mathbf{2^p \geq m + p + 1}$ (m = ডেটা বিট)।

m	4	8	16	32	64
p	3	4	5	6	7
কোড	(7,4)	(12,8)	(21,16)	(38,32)	(71,64)

অবস্থান: প্যারিটি বিট 1, 2, 4, 8 … (2-এর ঘাত) স্থানে, বাকিগুলোতে ডেটা। P1 দেখে 1, 3, 5, 7 · P2 দেখে 2, 3, 6, 7 · P4 দেখে 4, 5, 6, 7 (যে স্থানের বাইনারি সূচকে ওই বিট 1)।



প্রতিটি বৃত্তে জোড় প্যারিটি থাকবে

সমাধানকৃত উদাহরণ ✍️: জোড় প্যারিটি, ডেটা 1101 → D3 = 1, D5 = 1, D6 = 0, D7 = 1

১. P1 = D3⊕D5⊕D7 = 1 · P2 = D3⊕D6⊕D7 = 0 · P4 = D5⊕D6⊕D7 = 0

২. কোড-শব্দ (অবস্থান 1 → 7): **1 0 1 0 1 0 1**

৩. প্রাপ্ত 1000101: C1 = b1⊕b3⊕b5⊕b7 = 1 · C2 = b2⊕b3⊕b6⊕b7 = 1 · C4 = b4⊕b5⊕b6⊕b7 = 0

৪. সিনড্রোম C4C2C1 = 011 = **3** → বিট 3 উল্টাও → 1010101 ✓ (000 = ত্রুটি নেই)

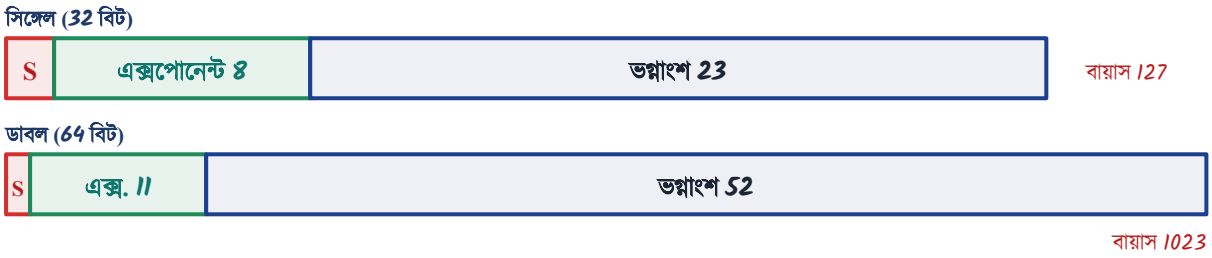
মনে রাখার কৌশল: "প্যারিটি বিট থাকে **2-এর ঘাতে**; সিনড্রোম হলো ভুল বিটের **ঠিকানা**।"

আইটি সংযোগ: সার্ভারের ECC RAM = SEC-DED (72,64): হ্যামিং(71,64) + ১টি সামগ্রিক প্যারিটি বিট। ইথারনেটে CRC-32; TCP/IP-তে চেকসাম।

১৫ ফিক্সড পয়েন্ট বনাম ফ্লোটিং পয়েন্ট

সংজ্ঞা: **ফিক্সড পয়েন্ট**: বাইনারি বিন্দু নির্দিষ্ট স্থানে (পূর্ণসংখ্যা, Q-ফরম্যাট)। **ফ্লোটিং পয়েন্ট**: মান = **চিহ্ন** × **ম্যান্টিসা** × $2^{\text{এক্সপোনেন্ট}}$, তাই বিন্দু "ভাসে"; সীমিত নির্ভুলতায় বিশাল পরিসর। ম্যান্টিসা 1.f আকারে থাকলে সংখ্যাটি **নরমালাইজড**, তাই শুরুর 1 সংরক্ষণ করতে হয় না (**লুকানো বিট, hidden bit**)।

১৬ IEEE 754 ফরম্যাট



$$\text{মান} = (-1)^S \times 1.F \times 2^{E\,-\,\text{bias}}$$

ফরম্যাট	বিট	S / E / F	বায়াস	নির্ভুলতা	সর্বোচ্চ ≈	ক্ষুদ্রতম নরমাল ≈
হাফ	16	1 / 5 / 10	15	11 বিট ≈ ৩ অঙ্ক	65 504	6.1×10^{-5}
সিঙ্গেল	32	**1 / 8 / 23**	**127**	24 বিট ≈ ৭ অঙ্ক	3.4×10^{38}	1.2×10^{-38}
ডাবল	64	**1 / 11 / 52**	**1023**	53 বিট ≈ ১৬ অঙ্ক	1.8×10^{308}	2.2×10^{-308}

E (সিঙ্গেল)	F = 0	F ≠ 0
0	±0	সাবনরমাল
1 … 254	নরমাল: 2^{E-127}	
255	±∞	**NaN**

মনে রাখার কৌশল: **"1-8-23, বায়াস 127; 1-11-52, বায়াস 1023"**। k-বিট এক্সপোনেন্টে বায়াস = $2^{k-1} - 1$। নির্ভুলতা = ভগ্নাংশ বিট + ১টি লুকানো বিট।

সমাধানকৃত উদাহরণ ১: −6.75 এনকোড ✍️:

১. $6.75 = 110.11_2 = 1.1011 \times 2^2 \rightarrow S = 1$

২. $E = 2 + 127 = 129 = 1000\ 0001$

৩. $F = 1011\ 0000\ \ldots\ 0$ (23 বিট; শুরুর 1 লুকানো)

৪. $1\ 10000001\ 10110000000000000000000 = \mathbf{C0D8\ 0000_{16}}$

সমাধানকৃত উদাহরণ ২: $4120\ 0000_{16}$ ডিকোড ✍️: $0\ 10000010\ 0100\ldots0 \rightarrow S = 0, E = 130 - 127 = 3$, মান = $1.01_2 \times 2^3 = 1010_2 = \mathbf{10.0}$

পরীক্ষার ফাঁদ: 0.1-এর সঠিক বাইনারি রূপ নেই (float 0.1 = $3DCC\ CCCD_{16}$), তাই double-এ $0.1 + 0.2 = 0.30000000000000004 \neq 0.3$: float কখনো == দিয়ে তুলনা কোরো না। বায়াস 127, **128 নয়**। ব্যাংকিং সফটওয়্যার টাকা রাখে পূর্ণসংখ্যায় (পয়সা) বা decimal-এ, float-এ কখনো নয়।

পরীক্ষার আগের রাতের পাতা

তথ্য-তালিকা:

$N = \Sigma\, d_i\, r^i \cdot n$ বিট $\longrightarrow 2^n$ বিন্যাস

অক্টাল = 3, হেক্স = 4 বিট প্রতি অঙ্কে

পূর্ণ অংশ: $\div$ r, ভাগশেষ নিচ থেকে

ভগ্নাংশ: $\times$ r, পূর্ণ অংশ ওপর থেকে

1's: উল্টাও · 2's: উল্টে + 1 = $2^n - N$

2's পরিসর $-2^{n-1} \ldots 2^{n-1} - 1$

SM, 1's পরিসর $\pm(2^{n-1} - 1)$, দুটি শূন্য

2's-এ MSB-এর মান = -2^{n-1}

$V = C_{n-1} \oplus C_n$

চিহ্ন সম্প্রসারণ: MSB কপি

r's = $r^n - N$ · (r−1)'s = $r^n - 1 - N$

(r−1)'s: এন্ড-অ্যারাউন্ড ক্যারি

BCD: > 9 বা ক্যারি $\longrightarrow$ + 0110

XS-3 = BCD + 0011

$G = B \oplus (B >> 1)$

হ্যামিং $2^p \geq m + p + 1$

শনাক্ত $d - 1$, সংশোধন $\lfloor (d-1)/2 \rfloor$

IEEE: $(-1)^s \times 1.F \times 2^{E - bias}$

MCQ-র জন্য সেরা ১২টি এক-লাইনের তথ্য

১. 8-বিট 2's পরিসর: −128 … +127

২. −45 (8-বিট 2's) = 1101 0011

৩. 1's complement ও SM-এ দুটি শূন্য

৪. অবৈধ BCD কোড: 1010 – 1111 (৬টি)

৫. স্ব-পরিপূরক: 2421, XS-3, 84-2-1

৬. 1011-এর গ্রে কোড = 1110

৭. ASCII = 7 বিট, 128টি অক্ষর

৮. 'A' = 65, 'a' = 97, '0' = 48

৯. প্যারিটি শুধু বিজোড় সংখ্যক ত্রুটি ধরে

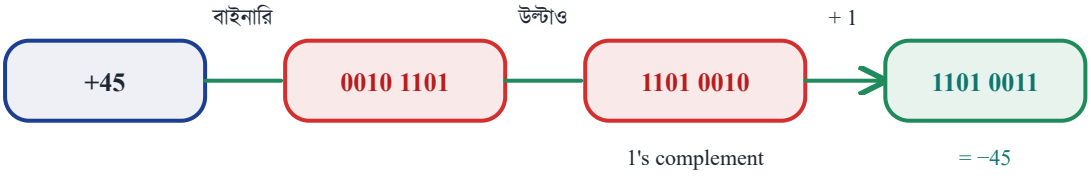
১০. হ্যামিং(7,4): ৩টি প্যারিটি বিট, d = 3

১১. ৮ ডেটা বিট $\longrightarrow$ ৪ প্যারিটি বিট (12,8)

১২. সিঙ্গেল float: 1-8-23, বায়াস 127

প্রধান ফাঁদ: 2's পরিসর অপ্রতিসম · ক্যারি ≠ ওভারফ্লো · ঋণাত্মক সংখ্যায় 0 বসালে মান বদলায় · BCD ≠ বাইনারি · BCD-তে 9 + 9-এ ক্যারির জন্য + 6 লাগে · গ্রে $\longrightarrow$ বাইনারিতে আগের **বাইনারি** বিট লাগে · ASCII 7-বিট · এক্সপোনেন্ট 255 = ∞/NaN · 0.1 বাইনারিতে সঠিক নয়

সব কৌশল একসাথে: অক্টাল ৩, হেক্স ৪ · প্রথম 1 পর্যন্ত কপি, বাকিটা উল্টাও · একটি ঋণাত্মক বেশি · 10's: শেষের 0 থাকে, প্রথম অশূন্য 10 থেকে, বাকি 9 থেকে · 9-এর বেশি বা ক্যারি $\longrightarrow$ 6 যোগ · গ্রে: প্রথমটি কপি, পাশাপাশি XOR · A-65, a-97, 0-48 · প্যারিটি বিট 2-এর ঘাতে, সিনড্রোম = ঠিকানা · 1-8-23 বায়াস 127



যে রূপান্তর প্রায় প্রতিটি প্রশ্নপত্রে আসে: 2's complement-এ −N

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. 8-বিট 2's complement সংখ্যার পরিসর:
ক) −127 … +127 খ) −128 … +127
গ) 0 … 255 ঘ) −128 … +128

২. 0110 1000-এর 2's complement:
ক) 1001 0111 খ) 1001 1001
গ) 1001 1000 ঘ) 0110 1001

৩. দশমিক 7-এর এক্সেস-৩ কোড:
ক) 0111 খ) 1010
গ) 1001 ঘ) 1011

৪. বাইনারি 1101-এর গ্রে কোড:
ক) 1001 খ) 1110
গ) 0110 ঘ) 1011

৫. ৮টি ডেটা বিটের হ্যামিং কোডে প্যারিটি বিট লাগে:
ক) 3 খ) 4
গ) 5 ঘ) 8

৬. BCD-তে 0111 + 0110-এর যোগফল:
ক) 0001 0011 খ) 1101
গ) 0000 1101 ঘ) 0001 1001

৭. IEEE 754 সিঙ্গেল প্রিসিশনের বায়াস:
ক) 127 খ) 128
গ) 1023 ঘ) 255

৮. কোন কোডটি স্ব-পরিপূরক?
ক) 8421 খ) গ্রে
গ) 2421 ঘ) ASCII

৯. 'a'-এর ASCII মান (দশমিকে):
ক) 65 খ) 48
গ) 32 ঘ) 97

১০. 2's complement যোগে ওভারফ্লো ধরা হয় যখন:
ক) $C_{out} = 1$ খ) দুটি ইনপুট < 0
গ) MSB-তে $C_{in} \neq C_{out}$ ঘ) ফল $= 0$

উত্তরমালা → ১-খ · ২-গ · ৩-খ · ৪-ঘ · ৫-খ · ৬-ক · ৭-ক · ৮-গ · ৯-ঘ · ১০-গ

লিখিত প্রশ্ন: জোড় প্যারিটির হ্যামিং(7,4) কোডে ডেটা 1011 এনকোড করো। প্রাপক পেল 0110001: ত্রুটি নির্ণয় ও সংশোধন করো। ১৬টি ডেটা বিটে কয়টি প্যারিটি বিট লাগবে? ৮ নম্বর

১ বিন্যাস P1 P2 D3 P4 D5 D6 D7; D3 = 1, D5 = 0, D6 = 1, D7 = 1 ১

২ P1 = 1⊕0⊕1 = 0 · P2 = 1⊕1⊕1 = 1 · P4 = 0⊕1⊕1 = 0 → কোড-শব্দ **0110011** ২

৩ প্রাপ্ত 0110001: C1 = b1⊕b3⊕b5⊕b7 = 0⊕1⊕0⊕1 = 0 · C2 = b2⊕b3⊕b6⊕b7 = 1⊕1⊕0⊕1 = 1 · C4 = b4⊕b5⊕b6⊕b7 = 0⊕0⊕0⊕1 = 1 ২

৪ সিনড্রোম C4C2C1 = 110 = **6** → বিট 6 ভুল → সংশোধিত 0110011, ডেটা 1011 ✓ ২

৫ ১৬ ডেটা বিট: $2^5 = 32 \geq 16 + 5 + 1 = 22$ ($2^4 = 16 < 21$) → **p = 5**, অর্থাৎ (21,16) কোড ১

পরীক্ষার আগে যাচাই ✓

- ✓ ভগ্নাংশসহ ভিত্তি রূপান্তর
- ✓ SM, 1's, 2's-এ −N; সব পরিসর
- ✓ ওভারফ্লো ও চিহ্ন সম্প্রসারণ
- ✓ BCD যোগ, বাইনারি ↔ গ্রে
- ✓ হ্যামিং(7,4), IEEE 754 সিঙ্গেল

লিখিতের জন্য আরও অনুশীলন:
① 10's complement-এ 3250 − 72532
② 1's complement-এ 1010100 − 1000011
③ BCD-তে 59 + 38
④ IEEE 754 সিঙ্গেলে −13.625